If you forward the encrypted mail to someone else, including your friend, the recipient will not be able to decode the mail. This is very useful when sending e-mails to a relation who loves to gossip and has an uncontrollable mailing list! A side effect is that unless you copy an encrypted mail to yourself, you can't see what you sent.

If you do not have the public key of a recipient and you give the request to encrypt the mail, Evolution will give you an error. However, if both the Kirans – the union leader and the CFO in our opening paragraph -- had a public key in the key ring, encryption is not going to prevent you from making a mistake.

An example of an application

Programs can make mistakes—and they do so consistently. They do not normally make silly mistakes unless, of course, programmed to do so.

Suppose you want to send salary slips to all your employees, and want each employee to view only his or her salary details. Every employee, on joining, can create a key pair and register the individual public key with the company. The admin staff need not manage these keys securely! In fact, they can freely distribute the public key to anyone who needs it -- for example, the bank where a salary account is opened.

Python has a module called *pygpgme*, which is a wrapper for the *gpgme*, GPG Made Easy, library. It is installed on Fedora, as *Yum* needs it. It lacks one small thing—documentation!

The *gpgme* library is documented, but seems to lack any tutorials or articles on how to get started with it.

The solution in such cases is to download the source. You can actually ignore the source code and search for the test cases. That can act as an excellent starting point.

Encrypting/decrypting a file

For your application, you need to be able to encrypt a file. So, try the following code:

```
import gpgme
infile = open('salary_slip.txt')
outfile = open('salary_slip.asc', 'w')
ctx = gpgme.Context()
ctx.armor = True
recipient_key = ctx.get_key('friend@example.com')
ctx.encrypt_sign([recipient_key], gpgme.ENCRYPT_ALWAYS_TRUST, infile,
outfile)
outfile.close()
infile.close()
```

The code is pretty straightforward. Open the two files and obtain the GPG context. The 'armor' option creates an ASCII file rather than a binary one. Obtain the key by using the recipient's e-mail address, then encrypt and sign the file by passing a list of the keys. The second option informs you that the keys should be trusted. You will be prompted for the passphrase while signing in, if you have specified one while creating your key.

The code for decrypting a file is even simpler:

```
import gpgme
infile = open('salary_slip.asc')
outfile = open('salary_slip.txt', 'w')
ctx = gpgme.Context()
ctx.decrypt(infile, outfile)
infile.close()
outfile.close()
```

gpgme will raise an exception in case decryption fails or the signature is not valid. The 'decrypt and verify' method will return a list of signatures. You may want to get some more information about the signatures.

Since there is only one key in your case, try the following code:

```
signing_key = ctx.get_key(sigs[0].fpr)
print signing_key.uids[0].name
print signing_key.uids[0].email
```

You get the key by using the fingerprint and then print the information you may need.

Let's suppose you just wanted to sign a text:

```
import gpgme
infile = open('salary_slip.txt')
outfile = open('salary_slip_signed.asc','w')
ctx = gpgme.Context()
ctx.armor = True
ctx.sign(infile, outfile, gpgme.SIG_MODE_CLEAR)
outfile.close()
infile.close()
```

You have chosen the clear sign mode so that the text is readable and the signature identifiable.

This will be enough for the moment. You can read the code in the tests subdirectory of the *pygpgme* source [*pypi.python.org/packages/source/p/pygpgme/pygpgme-0.1.tar.gz*] to learn more.

Mime and PGP

You are now in a position to combine encryption with the e-mail module so that the hard part is done by the application, and the user can access secure information very conveniently. The format for a Mime-encrypted message is described in *www.ietf.org/rfc/rfc3156.txt*.

Start with the various modules that need to be imported: